

README.md

Fuzzing in Go using GFuzz

Name: Jeremy Schiemann (61325) Professor: Prof. Martin Sulzmann

What is GFuzz?

GFuzz is a fuzzing framework specifically built to detect concurrency bugs in Go programs. Unlike traditional fuzzers that target program inputs, GFuzz focuses on non-deterministic execution schedules caused by concurrent goroutines. By systematically manipulating goroutine schedules, GFuzz can uncover edge-case bugs that are otherwise hard to reproduce.

How To

You can run the different examples by using the Makefile targets. This will alter the source code, so make sure to commit all changes beforehand and undo the changes after the run. Gfuzz will run indefinitely and try to find bugs until you abort the execution.

Output

After running Gfuzz the out directory will contain the result.

example:

```
[oraclert] selefcm timeout: 1500=== RUN    TestChannelBug
[oraclert] started
Should be buggy
[oraclert]: AfterRun
[oraclert]: AfterRunFuzz
[oraclert]: 1 selects
[oraclert]: CheckBugEnd...
End of unit test. Check bugs
-----New Blocking Bug:
---Primitive location:
/fuzz/target/hello_test.go:15
---Primitive pointer:
```

```
0xc000064ac0
```

```
-----End Bug
```

```
-----FOUND BLOCKING
```

```
---Stack:
```

```
goroutine 6 [running]:
```

```
runtime.DumpAllStack()
```

```
    /usr/local/go/src/runtime/myoracle_tmp.go:207 +0x85
```

```
gfuzz/pkg/oraclert.AfterRunFuzz(0xc0000a1b0)
```

```
    /usr/local/go/src/gfuzz/pkg/oraclert/oracle.go:369 +0xfd
```

```
gfuzz/pkg/oraclert.AfterRun(0xc0000a1b0)
```

```
    /usr/local/go/src/gfuzz/pkg/oraclert/oracle.go:322 +0x73
```

```
hello.TestChannelBug(0xc000d0500)
```

```
    /fuzz/target/hello_test.go:58 +0x325
```

```
testing.tRunner(0xc000d0500, 0x58b9e0)
```

```
    /usr/local/go/src/testing/testing.go:1193 +0xef
```

```
created by testing.(*T).Run
```

```
    /usr/local/go/src/testing/testing.go:1238 +0x2b5
```

```
goroutine 1 [chan receive]:
```

```
testing.(*T).Run(0xc000d0500, 0x5824dd, 0xe, 0x58b9e0, 0x48fe46)
```

```
    /usr/local/go/src/testing/testing.go:1239 +0x2dc
```

```
testing.runTests.func1(0xc000d0280)
```

```
    /usr/local/go/src/testing/testing.go:1511 +0x78
```

```
testing.tRunner(0xc000d0280, 0xc000c1de0)
```

```
    /usr/local/go/src/testing/testing.go:1193 +0xef
```

```
testing.runTests(0xc0000a198, 0x673960, 0x2, 0x2, 0xc1c792ebe3fc48c8, 0x6fca82589,
```

```
    /usr/local/go/src/testing/testing.go:1509 +0x305
```

```
testing.(*M).Run(0xc000c5790, 0x0)
```

```
    /usr/local/go/src/testing/testing.go:1417 +0x1eb
```

```
main.main()
```

```
    _testmain.go:45 +0x138
```

```
goroutine 8 [chan send]:
```

```
hello.TestChannelBug.func1(0xc000096580)
```

```
    /fuzz/target/hello_test.go:19 +0x9f
```

```
created by hello.TestChannelBug
```

```
    /fuzz/target/hello_test.go:16 +0xeb
```

```
--- PASS: TestChannelBug (0.30s)
```

```
PASS
```

At the top in the -----New Blocking Bug: section you can see it found a bug, the interesting part is ---Primitive location: . It shows you where the bug was found in the code. ---Primitive pointer: can be ignored.

Below is a stacktrace leading to the bug.

What to do with it?

This is a rather easy example, here is the code:

```
ch := make(chan int)
go func() {
    ch <- 1
}() //<-- this func is were GFuzz is pointing to

select {
case <-ch:
    fmt.Println("Normal")
case <-time.After(300 * time.Millisecond):
    fmt.Println("Should be buggy")
}
```

The problem is that channel `ch` is unbuffered. This means that a send (`ch <- 1`) will block until a corresponding receive (`<-ch`) is ready to accept the value.

The blocking can occur if:

- The send (`ch <- 1`) in the goroutine is not scheduled in time.
- The timeout is not triggered yet because the `select` is stuck waiting on the channel.

It is fixed easily by using a buffered channel instead:

```
ch := make(chan int, 1)
```

How does it work? (WIP)

This is the original select statement of the `hello-example` :

```
select {
    case <-ch:
        fmt.Println("Normal")
    case <-time.After(300 * time.Millisecond):
```

```

        fmt.Println("Should be buggy")
    }

```

There are 2 possible routes: a receive from the channel and a timeout. GFuzz changes the source code so that every branch will be on its own switch case.

```

switch oraclert.GetSelEfcmSwitchCaseIdx("/fuzz/target/hello_test.go", "17", 2) {
    case 0:
        select {
        case <-ch:
            fmt.Println("Normal")
        case <-oraclert.SelEfcmTimeout():
            oraclert.StoreLastMySwitchChoice(-1)
            select {
            case <-ch:
                fmt.Println("Normal")
            case <-time.After(300 * time.Millisecond):
                fmt.Println("Should be buggy")
            }
        }
    case 1:
        select {
        case <-time.After(300 * time.Millisecond):
            fmt.Println("Should be buggy")
        case <-oraclert.SelEfcmTimeout():
            oraclert.StoreLastMySwitchChoice(-1)
            select {
            case <-ch:
                fmt.Println("Normal")
            case <-time.After(300 * time.Millisecond):
                fmt.Println("Should be buggy")
            }
        }
    default:
        oraclert.StoreLastMySwitchChoice(-1)
        select {
        case <-ch:
            fmt.Println("Normal")
        case <-time.After(300 * time.Millisecond):
            fmt.Println("Should be buggy")
        }
    }
}

```

GFuzz will then generate a random index to choose which case and therefore which select branch is used. The Timeout will always contain the original select, so does the default. The default is used when GFuzz has not yet set an order for this operation.

Effects of different timeouts in selects

GFuzz cant find a blocking bug if the timeout in a select is higher than GFuzz' own timeout. If the timeout is lower, then there is no problem.

Number of runs

single run of the `hello_test.go` example will result in 6 runs:

- 1-init-hello.test-TestChannelBug-0
- 2-init-hello.test-TestWgBug-0
- 3-rand-hello.test-TestChannelBug-1
- 4-rand-hello.test-TestChannelBug-1
- 5-rand-hello.test-TestChannelBug-3
- 6-rand-hello.test-TestChannelBug-3

1 and 2 are the tests without any changes. Since 2 is already timing out in a normal run, it is ignored afterwards.

3-6 are the different runs of the `TestChannelBug` test.

ort_output 3:

```
{
  "tuple":{
    "4":1
  },
  "channels":{
    "/fuzz/target/hello_test.go:15":{
      "id":"/fuzz/target/hello_test.go:15",
      "closed":false,
      "not_closed":true,
      "cap_buf":0,
      "peak_buf":0
    }
  },
  "ops":[
    3,
    4
  ],
  "selects":[
    {
      "id":"/fuzz/target/hello_test.go:17",
      "cases":2,
```

```

        "chosen":0 // <--
    }
]
}

```

ort_output 4:

```

{
  "tuple":{
    "4":1
  },
  "channels":{
    "/fuzz/target/hello_test.go:15":{
      "id":"/fuzz/target/hello_test.go:15",
      "closed":false,
      "not_closed":true,
      "cap_buf":0,
      "peak_buf":0
    }
  },
  "ops":[
    3,
    4
  ],
  "selects":[
    {
      "id":"/fuzz/target/hello_test.go:17",
      "cases":2,
      "chosen":1 // <--
    }
  ]
}

```

ort_output 5 & 6: same as 3 & 4 but with higher timeout:

```

{
  "selefcms": {
    "sel_timeout": 1500, // <--
    "efcms": [
      {
        "id": "/fuzz/target/hello_test.go:17",
        "case": 1
      }
    ]
  },
}

```

```

    "dump_selects": true
  }

  {
    "selefcms": {
      "sel_timeout": 2500,  // <--
      "efcms": [
        {
          "id": "/fuzz/target/hello_test.go:17",
          "case": 0
        }
      ]
    },
    "dump_selects": true
  }

```

Big Project (Prometheus)

using Promethus release

- Project: Prometheus 2.26 (got go 1.16 compatibility, thank you ChatGPT for finding that version)
- Docker build: ~ 2.5 min (on Ryzen 5900X 12x 4.5 GHz)
- Runtime: Indefinetly, until you stop it.
- cmd: `./GFuzz/scripts/fuzz-git.sh https://github.com/prometheus/prometheus release-2.26 /mnt/c/Users/jerem/Documents/GitHub/go-fuzzing-seminar/out`

Console output while running is meaningless:

```

2024/12/04 17:06:45 {GoModDir:/fuzz/target TestFunc:[] TestPkg:[] TestBinGlobs:[] Or
TimeDivideBy:0 OracleRtDebug:false SelEfcmsTimeout:500 FixedSelEfcmsTimeout:false Scor
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38

```

```

/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38
/go/pkg/mod/go.uber.org/goleak@v1.1.10/testmain.go:53 +0x38

```

[...]

Output of `python3 ./GFuzz/scripts/analyze.py --gfuzz-out-dir ./out :`

ignore 2739-rand-github.com-prometheus-prometheus-notifier.test-TestHandlerQueuing-2

ignore 2743-rand-github.com-prometheus-prometheus-notifier.test-TestHandlerQueuing-2

ignore 2744-rand-github.com-prometheus-prometheus-notifier.test-TestHandlerQueuing-2

ignore 2745-rand-github.com-prometheus-prometheus-discovery-kubernetes.test-TestNode

total entries: 238 // <-- original test cases

total runs: 2745 // <-- runs of these tests with different orders

total duration (sec): 4021.0

average (run/sec): 0.68

total runs (without timeout): 2456

total duration (without timeout): 13726.0

average (run/sec): 0.18

bug statistics:

used (hours), buggy primitive location, gfuzz exec

0.19 h, /fuzz/target/discovery/manager_test.go:679,341-rand-github.com-prometheus-pro

- bug file
- ort_output

Conclusion

Pro

No additional code required
seems to find more niche bugs

Contra

No easy setup (quirks, no explanation whats wrong)
doesnt revert changes after run
config is ignored